

المحاضرة 6

بنى التحكم: الجزء الثاني
while – do while – for

د. سهيل الحمود
د. أسامه ناصر

2025-2026

مقدمة

- بنى التكرار **Loops** هي أدوات برمجية تسمح بتكرار تنفيذ جزء من التعليمات عدة مرات.
- نستخدمها عندما نحتاج:
 - تنفيذ تعليمة أكثر من مرة.
 - المرور على عناصر مصفوفة.
 - تنفيذ خوارزمية تعتمد على عدد محدد أو غير محدد من التكرارات.

الأنواع الأساسية لبنى التكرار

```
while(condition){  
    statements  
}
```

• حلقة while

```
do{  
    statements  
}while(condition);
```

• حلقة do while

```
for(initialization; condition; update){  
    statements  
}
```

• حلقة for

أولاً: حلقة while

الشكل العام

```
while(condition){  
    // تعليمة أو مجموعة تعليمات  
    statements  
}
```

آلية التنفيذ

1. يتم فحص الشرط **قبل** الدخول للحلقة.
2. إذا كان ناتج تقييم الشرط true عندها قم بتنفيذ كتلة الحلقة.
3. بعد التنفيذ قم بالعودة إلى بداية الحلقة و افحص الشرط مرة أخرى.
4. تستمر العملية حتى يصبح الشرط false.

مثال: طباعة الأعداد من 1 إلى 5

```
int i = 1;
while(i ≤ 5){
    cout << i << endl;
    i++;
}
```

1
2
3
4
5

مثال: حساب مجموع الأعداد من 1 حتى n

```
int n; cin >> n;
int sum = 0;
int i = 1;
while(i <= n){
    sum += i;
    i++;
}
cout << sum << endl;
```

ملاحظة مهمة

- إذا نسينا تحديث المتغير داخل الحلقة قد نحصل على **حلقة لانهاية**.

```
int n; cin >> n;
int sum = 0;
int i = 1;
while(i ≤ n){ // حلقة لانهاية
    sum += i;
    // i++; تحديث متغير الحلقة
}
cout << sum << endl;
```

ثانياً: حلقة do while

آلية التنفيذ

- يتم تنفيذ جسم الحلقة **قبل** فحص الشرط.
- يتم فحص الشرط بعد التنفيذ.
- تستمر الحلقة ما دام الشرط محققاً.
- أي أن جسم الحلقة ينفَّذ **مرة واحدة على الأقل** مهما كان الشرط.

الشكل العام

```
do{  
    تعليمة أو أكثر //  
    statements  
}while(condition);
```


مثال: قراءة رقم موجب فقط وحساب جذره التربيعي

```
#include <iostream>
#include <cmath>
using namespace std;

int main(){
    double x;
    do{
        cout << "Enter positive number: ";
        cin >> x;
    }while(x ≤ 0);
    cout << sqrt(x) << endl;
    return 0;
}
```

الفرق بين while و do while

do while	while	الخاصية
بعد التنفيذ	قبل التنفيذ	توقيت فحص الشرط
تنفذ مرة واحدة على الأقل	قد لا تنفذ أبداً	تنفيذ كتلة التعليمات

ثالثاً: حلقة for

الشكل العام

```
for(initialization; condition; update){  
    // تعليمات  
}
```

آلية التنفيذ

1. تنفيذ الجزء **initialization** لمرة واحدة.
2. فحص الشرط **condition**.
3. إذا تحقق الشرط → تنفيذ جسم الحلقة.
4. تنفيذ **update** ثم العودة للشرط.

مثال: طباعة الأعداد من 1 إلى 10

```
for(int i=1; i ≤ 10; i++){  
    cout << i << endl;  
}
```

مثال: مجموع أول n عدد

```
int n; cin >> n;
int sum = 0;
for(int i=1; i<=n; i++){
    sum += i;
}
cout << sum;
```

مقارنة بين أنماط الحلقات الثلاث

for	do while	while	
داخل تعريف الحلقة	خارج الحلقة غالبًا	خارج الحلقة	مكان تهيئة المتغير
قبل التنفيذ	بعد التنفيذ	قبل التنفيذ	توقيت فحص الشرط
حلقات ذات عدد معروف	قراءة مدخلات حتى تصبح صحيحة	حلقات ذات عدد غير معروف	أمثلة استخدام

مثال: طباعة الأعداد الزوجية من 2 إلى 20 بثلاث طرق

باستخدام `while`

```
int i = 2;
while(i ≤ 20){
    cout << i << " ";
    i += 2;
}
```

باستخدام `do while`

```
int i = 2;
do{
    cout << i << " ";
    i += 2;
}while(i ≤ 20);
```

باستخدام `for`

```
for(int i=2; i ≤ 20; i+=2){
    cout << i << " ";
}
```

تحذير: الحلقات اللانهائية

حلقة لا تنتهي باستخدام **while** :

```
int i = 1;
while(i > 0){
    cout << i;
} // لا يتغير أبداً i!
```

حلقة لانهاية باستخدام **for**

```
for(;;){
    // تعمل للأبد
}
```


مثال: حساب متوسط العلامة لمجموعة من المواد

خطوات الحل

- قراءة عدد المواد
- تكرار القراءة داخل حلقة بحسب العدد المدخل
- جمع كل الدرجات داخل متغير sum
- بعد انتهاء الحلقة نحسب المتوسط
- المتوسط = مجموع الدرجات / عدد المواد
- طباعة الناتج

مثال: حساب متوسط العلامة لمجموعة من المواد باستخدام حلقة for

```
int n;  
cout << "Enter the number of modules: ";  
cin >> n; // عدد المواد  
  
double sum = 0;  
for(int i = 0; i < n; i++){  
    int grade;  
    cin >> grade;  
    sum += grade;  
}  
  
double average = sum / n;  
cout << "Average = " << average << endl;
```

مثال: حساب متوسط العلامة لمجموعة من المواد باستخدام حلقة while

```
int n;
cout << "Enter the number of modules: ";
cin >> n; // عدد المواد

int i = n;
double sum = 0;
while(i > 0){
    int grade;
    cin >> grade;
    sum += grade;
    i--;
}

double average = sum / n;
cout << "Average = " << average << endl;
```

مثال: حساب متوسط العلامة لمجموعة من المواد

باستخدام حلقة while

استخدام الشرط و تغيير عداد الحلقة في خطوة واحدة

```
int n;
cout << "Enter the number of modules: ";
cin >> n; // عدد المواد

int i = n;
double sum = 0;
while(i--){
    int grade;
    cin >> grade;
    sum += grade;
}

double average = sum / n;
cout << "Average = " << average << endl;
```

التحكم بسير الحلقات

باستخدام `break` و `continue`

- في الحلقات التكرارية (`for` – `while` – `do while`) نحتاج أحيانًا لتغيير مسار التنفيذ.
- توفر C++ أداتين مهمتين:
- `break` → للخروج من الحلقة بالكامل.
- `continue` → لتجاوز المرور الحالي والانتقال إلى المرور التالي.

تعليمة break

- توقف الحلقة فورًا.
- تخرج مباشرة إلى أول سطر بعد جسم الحلقة.
- تستخدم بشكل مرافق لتعليمات شرطية

الشكل العام

```
while(condition){  
    statements;  
    if(condition)  
        break;  
    statements;  
}
```

مثال على break

النتيجة: 1 2 3 4 5

```
for(int i = 1; i ≤ 10; i++){  
    if(i == 6) break;  
    cout << i << " ";  
}
```

لماذا؟

- عند $i = 6$ يتم تنفيذ break و عندها يتم إنهاء الحلقة و الانتقال إلى التعليمة التالية بعد الحلقة.

مثال: حساب متوسط العلامة لمجموعة من المواد باستخدام حلقة while و break

عدد المواد غير محدد من البداية, استمر بالقراءة حتى تحصل على علامة خارج المجال

```
int n = 0;
double sum = 0;
do {
    double grade;
    cout << "Enter the grade: ";
    cin >> grade;
    if (grade < 0 || grade > 100)
        break;
    sum += grade;
    n++;
} while (true);
cout << "Number of modules: " << n << endl;
cout << "Average = " << sum / n << endl;
```


تعليلة continue

الوظيفة

- لا تخرج من الحلقة.
- تتجاوز بقية تعليمات المرور الحالي.
- تنتقل مباشرة إلى بداية الحلقة.
- تستخدم عادة مع تعليمات شرطية

الشكل العام

```
for(initialization; condition; step){  
    statements;  
    if(condition) continue;  
    statements; //  
}
```

مثال على continue

طباعة الأعداد الفردية فقط

```
for(int i = 1; i ≤ 10; i++){  
    if(i % 2 == 0)  
        continue; // تجاوز العدد الزوجي  
    cout << i << " "; // يطبع العدد الفردي فقط  
}
```

// output: 1 3 5 7 9

النتيجة: 1 3 5 7 9

مثال على break و continue

طباعة الجذر التربيعي لعدد

- حلقة لانهاية
- اقرأ العدد
- اذا كان العدد سالبا أهمل بقية التعليمات لهذا المرور و ارجع إلى بداية الحلقة
- إذا كان العدد يساوي صفرا قم بالخروج من الحلقة نهائيا إلى نهاية البرنامج
- إذا كان العدد موجبا احسب و اطبع الجذر التربيعي له ثم ارجع إلى بداية الحلقة

مثال على break و continue

طباعة الجذر التربيعي لعدد

```
while(true){  
    cout << "x ? ";  
    double x; cin >> x;  
  
    if(x < 0) continue; // تجاهل القيم السالبة  
    if(x == 0) break; // اخرج من الحلقة عند الصفر  
  
    cout << "sqrt(" << x << ")=" << sqrt(x) << endl;  
}
```

الفرق بين break و continue

continue	break	الخاصية
تجاوز بقية تعليمات المرور الحالي فقط	الخروج من الحلقة بالكامل	تأثيرها
إلى بداية الحلقة	إلى أول سطر بعد الحلقة	الانتقال
تخطي تعليمات أو حالات معينة	إيقاف التكرار	الاستخدام

الحلقات المتداخلة (Nested Loops)

- هي استخدام حلقة داخل حلقة أخرى.
- تُستخدم عندما نريد تنفيذ تكرار متعدد الأبعاد مثل:
 - الطباعة على شكل جداول.
 - التعامل مع مصفوفات ثنائية.

الشكل العام للحلقات المتداخلة

```
for(int i = 1; i ≤ n; i++){  
    for(int j = 1; j ≤ m; j++){  
        // تعليمات تعتمد على  
        // i و j  
    }  
}
```

- الحلقة الخارجية تتحكم بعدد الصفوف.
- الحلقة الداخلية تتحكم بعدد الأعمدة.

مثال: طباعة جدول 3×3

```
for(int i = 1; i ≤ 3; i++){  
    for(int j = 1; j ≤ 3; j++){  
        cout << 10 * i + j << " ";  
    }  
    cout << endl;  
}
```

النتيجة:

```
11 12 13  
21 22 23  
31 32 33
```


مثال: طباعة أرقام الصف + العمود

```
for(int i = 1; i ≤ 3; i++){  
    for(int j = 1; j ≤ 3; j++){  
        cout <<"(" << i << ", " << j << ") ";  
    }  
    cout << endl;  
}
```

النتيجة:

```
(1,1) (1,2) (1,3)  
(2,1) (2,2) (2,3)  
(3,1) (3,2) (3,3)
```

مثال: طباعة مربع نجوم

الناتج:

```
int n = 5;
for(int i = 1; i ≤ n; i++){
    for(int j = 1; j ≤ n; j++){
        cout << "*";
    }
    cout << endl;
}
```

```
*****
*****
*****
*****
*****
```

مثال: طباعة مثلث نجوم

الناتج:

```
for(int i = 1; i ≤ 5; i++){  
    for(int j = 1; j ≤ i; j++){  
        cout << "*";  
    }  
    cout << endl;  
}
```

```
*  
**  
***  
****  
*****
```

مثال: جدول الضرب باستخدام حلقات متداخلة

```
for(int i = 1; i ≤ 10; i++){  
    for(int j = 1; j ≤ 10; j++){  
        cout << i * j << "\t";  
    }  
    cout << endl;  
}
```

استخدام break داخل حلقات متداخلة

```
for(int i = 1; i ≤ 5; i++){  
    cout << "row " << i << " → ";  
    for(int j = 1; j ≤ 5; j++){  
        if(j = 3) break;  
        cout << j << " ";  
    }  
    cout << endl;  
}
```

- break توقف الحلقة الداخلية فقط

- خرج البرنامج:

```
row 1 → 1 2  
row 2 → 1 2  
row 3 → 1 2  
row 4 → 1 2  
row 5 → 1 2
```

استخدام continue داخل حلقات متداخلة

```
for(int i = 1; i ≤ 5; i++){  
    for(int j = 1; j ≤ 5; j++){  
        if(j = 2) continue; // يتجاوز العمود الثاني  
        cout << j << " ";  
    }  
    cout << endl;  
}
```

تمارين

1. احسب مجموع الأعداد بين عددين صحيحين n و m .
2. احسب القاسم المشترك الأعظم لعددين صحيحين باستخدام خوارزمية اقليدس
3. تبديل ترتيب الخانات العشرية لرقم صحيح
4. احسب قيمة العامل $n!$ لعدد معين
5. اطبع جدول الضرب من 1 إلى 10 باستخدام حلقة متداخلة.
6. تحديد إذا كان العدد أوليا